

Image

CHAPTER 10

Secure Software Development Lifecycle

This chapter is primarily intended to help you stop a *software insecurity disaster* before it happens. A *secure software development lifecycle* (SSDLC) includes a wide range of security processes and activities throughout the design, development, and live execution of your software. None of this is easy or scientific and although many organizations use similar processes to build secure software, the intricacies and culture of most organizations are very different, even among organizations that claim to use the same software building process. While process is often one of the “softer” topics in secure coding, it is also one of the most important—especially for large organizations.

The more mature your team becomes at implementing a secure software development process, the earlier security bugs in your software can be addressed. Addressing possible vulnerabilities earlier in the lifecycle also saves money and brings down the overall cost to build a secure product. It is also critical to ensure that “fixed” security bugs are not reintroduced by rolling back code or because of a new programmer who is less skilled at secure development. Also, your software will be safer because fewer security bugs should really lead to fewer security incidents in your live software. Even when and if you do get breached, a mature SSDLC includes good incident response and has prepared software and other processes to make handling and managing a hacking incident a well-thought-out process.

Averting Disaster Before It Starts

What is more expensive? Investing in software security and the construction of secure software from the start? Or is it more expensive to pay for the recovery from a hacking event and the retrofit needed to prevent future attacks? It really can go either way. Are you feeling lucky, punk? In one camp, we have the cry of “Let’s get back to real engineering and be more careful about the security properties we’re building in to our software.” This camp thinks it is much more costly to recover from an exploit than to do the right thing in the first place. In the other camp, we have the cry of “Move fast and we can worry about security later or just pay for cybersecurity insurance or just wait to get hacked and pay for it then.” Really, it could go either way. Before making that decision, you have to make a clear inventory of your site and assess the overall risk faced by your application. In order to do this you must consider your assets, the possible motivations of an attacker, and the potential outcomes of a security breach.

Assets

Assets are things that are accessible through your website and might be of some value to an attacker. The types of assets your website handles will help you in understanding the motivations of a theoretical attacker. The following list is a sample of assets that an attacker might want:

• • • • • Image Money

• • • • • Image Credit card numbers

• • • • • Image Trade secrets

Image Personally identifiable information (PII)

Image Passwords or encryption keys

Image Access to back-end systems

In addition to these assets, as the Internet of Things advances, you will need to consider actual physical devices that your web application can access. We have already seen presentations from security researchers on remotely hacking in to webcams and toys. The list of connected devices is growing to include cars, industrial equipment, and many other things we interact with on a daily basis.

Motivations

If your website has valuable assets or moves money, you will almost certainly be facing highly motivated and highly skilled attackers, and you had better invest in security. Likewise, we are seeing state sponsored hackers go after industrial and trade secrets. Why would a hacker want to attack your site? Consider the following possibilities:

Image Money!

Image Information

Image To impersonate or stalk your users

Image Shame you before the world (hacktivism)

Image Personal/organizational vendetta (such as Anonymous vs. HBGary)

Image LULZ (such as Anonymous vs. Scientology)

Image Just to prove they can

Outcomes

What happens if you get hacked? If all you are doing is providing a forum for people to post cute groundhog pictures, then maybe it doesn't matter so much. But what happens when you have the biggest groundhog site on the Web and a security incident costs you thousands of dollars per day in lost advertising? The following are some possible outcomes of a hack:

Image Money stolen

Image Downtime causes lost revenue

Image Trade secrets lost

Image Regulatory fines/penalties

Image Lose the trust of your users

Beyond mere money, there is another factor to consider—consumer confidence. Once you lose the trust of your users it is very difficult to win it back. Simply fixing the issues and increasing your security budget is not enough. Money cannot buy back lost confidence overnight. Long after the software is remediated, concerns and questions can linger in the minds of users for years. The

currency of consumer confidence is long and steadfast dedication to your security goals. It is better never to lose it and the cost to restore it is astronomical.

Image

TIP

When you get hacked, the “cost” from the event might not be monetary. Lost trust or lost morale can cause significant harm to your organization and is often hard to quantify. Estimating the true risks to your applications can be difficult. However, if you can quantify it to any extent, it can help you make a business case for more security resources. The fact is that almost nobody says security is not important at all, but just how much to apply, and where and when to apply it, can be difficult to know.

Team Roles for Security

Building secure software, especially complex software for a large organization, is a team sport. Many individuals need to collaborate effectively (often under great duress) in order to efficiently and accurately produce secure software. The main roles to consider are discussed in the sections that follow.

Role: Security and Software Architect

One of the most difficult roles in a software team is the role of the security *and* software architect. Not only does this individual need to be incredibly astute in his or her understanding of the complications of building software, but he or she also needs to be up-to-speed on software threats as well as critical defense patterns. This individual also needs to be able to communicate effectively with “the business” to ensure that proper business requirements can be translated into a working piece of software. The knowledge that this one role needs is so significant, most large companies have architecture “councils” or some other mechanism where more than one person serves this role. Hit any job bank with a search for “software architect” and you will see requirements that include:

Image Deep mastery of modern web technologies even before they are invented

Image Active engineer with experience building highly scalable web, web service, and mobile applications that have zero bugs

Image Ability to build, communicate, and measure complex architectural designs while everyone else on your team has different ideas and throws stones at your ideas

Image Team leadership qualities that support a software engineering culture that will find and retain skilled talent, even when you really can’t pay anyone anything close to the value of their skill set

Image A passion for creating, building, and deploying extremely complex software even when management throws unrealistic and arbitrary deadlines at you

Image Actively coding 20–40 hours a week while still doing other aspects of your job that are more visible to upper management

Image A mastery of application security defense techniques and the ability to effectively mind-meld with all developers on your team to impart that knowledge in real time

Image A masters degree or higher in computer science or similar and hopefully no longer paying off your six-figure college loan

Image Fifteen years of active coding experience

Image Ability to hide the often sociopathic tendencies of your best developers from upper management

Image Expertise in object-oriented analysis and design“such that you wake up each morning after a night of restless sleep with scattered notes and UML diagrams all over your bedroom

Image Writing skills so amazing that your architectural analysis documents could be published into a Hollywood movie starring at least two A-list actors

Image A belief that working 60 hours is a “œlight week”

We may have added a bit of snark in the preceding items, but clearly, this role is hugely important to the process. From a secure SDLC point of view, a software architect must be responsible for actionable secure coding standards and software tools/libraries that will enable a developer to efficiently write secure code. If your architect is more of an “œarmchair architect” and not in the mix actively helping set the standards and practices needed for secure software, then the chance of achieving your security goals is very low.

An architect can make or break a company. Companies, choose your architect wisely.

A company can make or break an architect. Architects, choose your company wisely.

Role: Project Manager

*PMs want quality (and sometimes security) to be changeable, so they can sacrifice it when project runs late in order to make the schedule.*¹

“Robert David Graham

The relationship between project managers and engineering staff is often tenuous. However, project managers play a crucial role in ensuring that budgets, time commitments, promised features, and necessary security engineering are achieved. When push comes to shove and commitments are looming, project managers often have the unfortunate role of deciding what gets cut. Often, PMs will decide to cut things that are less noticeable to management and customers. For example, cutting a major feature would be visible to all, but not polishing a certain feature might sneak through the cracks. And security vulnerabilities, especially nonfunctional (non-visible quality-centric) requirements, are something that can easily be pushed back in the schedule without management or users noticing. The problem is that your adversaries will notice. Or that big client or company that wants to acquire your IP will insist on your software passing a security review before signing on the dotted line. This is why it is so crucial to budget time and resources for security activities, both before and during the development process. A good project manager will make sure that this often “œinvisible” need is clearly accounted for in a visible way before the team.

Role: Developer

While managers are managing and architects are architecting, someone needs to do the actual work of writing and maintaining the actual code. A great software developer is perhaps the most

important individual when it comes to building secure software. When times are tough and budgets are lean, the best developers are the last to go. When emergencies happen and fixes are quickly needed in software, your best developers are the first ones you call. Developers, like any beautiful and delicate flower, need care, feeding, and occasional sunlight.² A talented developer who is uneducated about security, however, can be a disaster. They can rock out new code and features very rapidly, but will also introduce many vulnerabilities in the process! That is why it is critical that you buy several copies of this book and give one to each of the developers on your team. At the very least, ensure your developers already have the secure coding knowledge they need early on in a project's lifecycle. We recommend that developers should at least be trained on the secure coding techniques that are relevant for their project and technology stack, and on how to use the tools and methods that were so painstakingly specified by your security architects. We have seen some companies that only give their developers this type of training after a breach or after security vulnerabilities are discovered in software. This is unfortunately far too late in the game. By the time a breach is actually discovered, training is scheduled and executed, and the bug is fixed, an attacker could be running roughshod through your application for weeks! The perfect time to train your developers about software security is at the beginning of the project—before the coding phase has begun.

Role: QA Tester

Most QA professionals are very well suited to verify the functional requirements of software. These are often the least technical members of your team because even non-techies can be effective QA professionals. At first glance, a QA tester just needs to be able to use and master your web application so it can be thoroughly tested for bugs. Admittedly, there are several levels of QA staff. Advanced QA staff leverage a variety of automation to assist in their task. The best QA staff are developers themselves, writing unit tests and other automation, and are fully integrated into the development staff.

Some software methodologies, such as DevOps, skip QA altogether and roll that process into development activities such as unit testing. One big mistake that many companies make is to believe that a standard QA professional can be turned into a penetration tester or other security professional overnight. While QA professionals are often the least technical members of your team, security professionals and penetration testers need to be some of the most technically advanced members of your team. It is not simple (nor sometimes even possible) to magically turn your QA staff into professional security testers.

Professional Security Tester

A penetration tester is a professional “good guy” hacker who ethically hacks your website and informs you of any vulnerabilities early in order to give you a chance to fix them before they “go live” and are exploited. This is one of the oldest information security professions and the need for these professionals has seen a huge upsurge over the last several years. Pentesters have the ability to dive deep into your website and discover vulnerabilities that automated tests are often not able to find, such as business logic (custom business processes) or sometimes access control vulnerabilities. These individuals are not inexpensive and are in high demand. Many pentesters work for small consulting firms and avoid working for large companies or firms. This is a challenge because the need for professional testing is far beyond the supply of experts in this field, and many new professionals are entering the security testing field without the right expertise. Perhaps the best way to create penetration testers is to start with experienced software developers and retrain them in the art and science of penetration testing. The insight gained from knowing how to build websites really helps when attempting to exploit them.

How do you know if the pentester you hired is thorough enough, skilled enough, and professional enough to be the right tester for your team? You hire an outside security firm and cross-check results against your own in-house team. If you had a health issue you would not dream of treating yourself and self-prescribing. At the very least you would seek a second opinion before consenting to expensive and potentially dangerous surgery. Why should security be any different? An outside firm can (and should) review your security program and occasionally test your applications for holes your own team might have missed. More eyeballs are better to ensure the most comprehensive results are produced. Using an outside firm also helps mitigate threats from malicious actors *inside* your own company.

An important point to remember is that a security report with no negative results does not mean that your application has zero vulnerabilities. No penetration test can ever certify that. Even if some vulnerabilities were reported, chances are that there were others that were never discovered, either due to time constraints, negligence, or sheer obscurity. For those preferring an automated means of ensuring complete test coverage, the OWASP Code Pulse Project provides you with a way to monitor penetration tester activities and other black box (outside-in testing) activities against your website during the security testing phase.³

Security Throughout the Application Lifecycle

There are many software development lifecycle modalities, but they all share the same basic elements: design, develop, test, deploy. Mix all four elements and shake well (do not stir), and whoosh, you have DevOps, Agile, AgileOps, Extreme Waterfall, and so forth. Regardless of your chosen methodology, you still need to prepare for security, you still need to build with security in mind, and you still need to test for security. And hopefully, you are also deploying and handling runtime operations in a secure fashion. The fact that these steps need to be done does not change whether you are scrumming or still stuck in a waterfall model.

The more agile approaches often favor less documentation and more action. Unfortunately, this generally does not give the security architect anything formal to review and evaluate. Development may be significantly under way by the time the security architect is even engaged.

Until the day that every software developer has the knowledge of a security professional, any code delivered without security review incurs risk. Unreviewed code places the product in an indeterminate state where risk is impossible to evaluate.

For a more formal review of secure software development lifecycles, please check out BSIMM (Building Security In Maturity Model) and OpenSAMM (Software Assurance Maturity Model) from the OWASP Foundation.⁴ BSIMM is a study of many large organizations and the steps they take in trying to build secure software. This study does not attest to the success of these methods, but the study and various steps described are very interesting, especially if you are a large company building very complex software projects. OpenSAMM is an open framework to help your organization build a custom software development lifecycle based on the specific security risks facing your software. Both of these models were birthed from the same group that forked into two different but interesting and useful paths. Both of these are worth reviewing if you are seeking information on more formal secure SDLC models.

Please note, the various secure-SDLC steps, which we will describe shortly, are not always meant to be accomplished in sequential order. Certain activities can and should be implemented in a continuous fashion throughout the software development lifecycle, depending on the type of software methodology or religion your organization subscribes to.

Security in the Software Development Lifecycle

Now that you have built your team, it is important to consider security at every stage of the software development lifecycle. At each stage, consider how you can best make use of the resources at your disposal.

Business Requirements

What does the business want out of the software you are tasked to build? Even “agile” teams need to start with some kind of business requirements. A good place to start in the construction of secure software is to capture the needs of the business. There are several critical aspects to a secure SDLC that are best handled at this stage. These include setting realistic expectations about how much time and budget can be allocated for security activities. If your company does not already have resources such as developer security training, static and dynamic analysis, and penetration testers, now is the time to plan for those.

Access control and authentication requirements should be determined at this stage. The questions your architects should be asking include: Is this a multi-tenant system? Do we have existing authentication services we need to integrate with? Do we need to support federation with other partners? Although it’s often preferable to handle technical security requirements at a later phase, capturing business requirements that affect authentication and authorization is critical at this stage because changing either of these mechanisms “late in the game” can be incredibly expensive.

Standards/Compliance

Some of the most visible security drivers for upper management are the various compliance standards such as PCI-DSS (credit card management), HIPAA (health care information management), Gramm-Leach-Bliley (financial information management), and various international privacy laws, especially in Europe. While these various standards are often criticized by security professionals as not being technically accurate or complete (and rightfully so), they are incredibly powerful business drivers for security that have painful financial consequences if not followed.

PCI-DSS is the payment card industry data security standard. If you are processing, storing, or using credit cards in your software, this standard applies to you. For more information, see https://www.pcisecuritystandards.org/security_standards/. This standard defines which parts of the credit card information can be stored, and how those parts need to be encrypted when stored or transferred over a network.

HIPAA stands for the Health Insurance Portability and Accountability Act of 1996. HIPAA forces health care providers (and companies managing health care information of their workers) to ensure that certain privacy requirements are met. If a breach of health care data occurs, HIPAA defines rules for notifying patients whose data may have been compromised. The notification requirements provide a strong incentive for upper management to plan for security in advance, rather than suffer a potentially embarrassing disclosure if an incident occurs.

The Gramm-Leach-Bliley Act is a massive financial reconstruction law that allowed large financial institutions to conduct various financial activities under one roof. That is all well and good for the bankers, but the part of the act that we as security professionals care about is that companies that offer consumer-centric financial services have to publicly disclose their information-sharing practices and take steps to protect consumer data.

The European Union is considering various regulations in Europe to ensure the privacy of its citizens. These potential regulations are some of the strongest ever to be proposed.⁵ The new regulations, if passed, will provide for a fine of up to 250,000 EUR, or up to 0.5 percent of a company's annual worldwide turnover, to anyone who does not protect personal data. Wince. Now *that* is going to get an executive's attention!

Technical Security Requirements for Developers

At the end of, or in sync with, design processes, it's time to tell the developers just what needs to be coded. Developers need to be aware of high-level business requirements and often this is where most companies stop. However, prescriptive technical security requirements can be a powerful force to corral developers into caring about security. Clear requirements, such as "ensure that all database queries are parameterized with bound variables" can have a powerful positive effect. Less clear requirements such as "make sure access control is implemented" often leave developers scratching their heads, or moving down a path that is not what you intended in the first place.

Technical requirements are often broken down into two or more major categories: functional and nonfunctional requirements. Functional requirements are visible requirements that can often be tested by a standard QA professional. From a security point of view, these visible requirements may look like: "Ensure that users provide their current password during the change password process" or "Ensure that the forgot password workflow follows these X steps" or "Make sure that only users with administrative entitlements are allowed to visit the main admin page." The benefit of this level of functional security requirement detail is that these same requirements can be used not only to manage developers, but also to help build a detailed security testing plan that can leverage existing QA staff.

Nonfunctional requirements are a different matter. These are the hidden "quality-centric" requirements that do not translate easily in a clear testing plan for QA staff. A good example of a nonfunctional requirement that most developers have already heard is "make it fast." Over time, performance requirements have matured from vague allusions to speed, to maximum page rendering times, transactional throughput, and total concurrent users, and sophisticated tools have been developed to ensure that these goals are met.

Nonfunctional requirements are built for developers, such as "Make sure that all queries are parameterized" or "Make sure that passwords are stored using salted PBKDF2-SHA256 with 2048-bit keys and at least 150,000 iterations." It's often not realistic to ask QA to verify these requirements. Often, you need a security professional to accurately verify these. It is easy to go overboard with nonfunctional requirements so a proper balance in detail is required. We want to ensure that the developer accomplishes the right security goal without forcing him down a path that may not be appropriate for the specific problem he is facing. However, good nonfunctional security requirements should at least force a conversation between developers and security architects about what security controls are needed, and sometimes this is all you need to move developers in the direction of secure software. Once developers understand what security controls are required and why they are important, most are intelligent and diligent enough to see it through.

A good open source resource to help you start down the path of nonfunctional (and other) security requirements is the OWASP Secure Coding Quick Reference Guide and the OWASP Application Security Verification Standard (ASVS).⁶ Even though the OWASP ASVS standard is for "assessment," we have also found it to be helpful when building security requirements for developers.

Implement Security Controls as Code Is Developed

One of the most important technical tasks a security architect can lead is to help build a secure coding library for that application (and hopefully for the organization). Do not write your security controls from scratch! Also, we advise that you do not depend directly on a third-party library for security. When you can, wrap those libraries beneath your own security API so different implementations can be swapped out for new ones over time. Very often, we ask developers to build security into their applications without giving them the tools or training to do so. This is a recipe for disaster.

The paragraphs that follow include several secure coding libraries or frameworks that may assist you in building your own security stack. Apache Shiro⁷ is an authentication and authorization framework and more. We are big fans of Shiro for its attribute-based access control. Apache Shiro is a “heavy” library and is a big investment in time to use. This will be the most difficult to “wrap” beneath our own Security API.

For cross-site scripting defense, we like the good ol’™ JavaServer Pages Standard Tag Library,⁸ the OWASP Java Encoder for output encoding XSS defense, the OWASP Java HTML Sanitizer for untrusted HTML sanitization, and the Java JSON Sanitizer for outbound or inbound JSON sanitization. For applied cryptography, the Google Keyczar⁹ library is leaps above the rest in this space and can be used in conjunction with the open source Bouncy Castle¹⁰ library for a more modern crypto provider. We also are fans of HeadLines,¹¹ a collection of HTTP response header security APIs! And last, we use Logback,¹² one of the most modern and attributable logging frameworks.

These components can and should be used to build your own security stack. You will make your developers’™ lives easier by making high-quality and comprehensive security libraries available.

Third-Party Dependencies

Every Java developer depends on third-party libraries to build their application. But to what extent can we trust these libraries? Jeff Williams and Arshan Dabirsiaghi from Aspect Security released a report, *The Unfortunate Reality of Insecure Libraries*, in March 2012.¹³ Their research analyzed over 113 million downloads of the 31 most popular open source Java frameworks and libraries available from the central repository. The report concluded that:

• • • • • Image Applications typically use 30 or more libraries.

• • • • • Image 88 percent of code in today’s™ applications comes from libraries and frameworks.

• • • • • Image 26 percent had known vulnerabilities.

• • • • • Image Most vulnerabilities are undiscovered.

It’s™ amazing just how much Java code we depend on as “trusted” without any analysis! There is a prevailing view that open source code is better or more secure because more people are looking at it. We often assume that “the community” has already taken care of any issues, just by virtue of many people working on it, but this is not always the case. As we have already discussed, many developers are great at adding features, but lack training in secure development. Don’t™ allow any unreviewed third-party libraries into production code!

OWASP Dependency Checker for Java The OWASP Foundation has a free tool called DependencyCheck that will aid in reviewing your third-party Java libraries and compare those jars against published vulnerabilities.¹⁴ This free tool will help you find publicly disclosed vulnerabilities in third-party code, for free!

jdeps is a command-line utility that is included with Java 8 Standard Edition. It is used to identify library dependencies. jdeps is useful from a security perspective because it identifies dependencies in third-party libraries that may negatively impact the ability to upgrade.

Test That Security Controls Have Been Properly Implemented

Testing software for security can be an expensive and difficult process, especially late in the software development lifecycle. It's prudent to use a combination of static code analysis (SAST), dynamic analysis against a running application (DAST), penetration testers to evaluate business logic and business-specific vulnerabilities (pentesting), and manual deep-dive code review. These techniques can be both expensive and time-consuming. It's prudent to use these testing methodologies based on the severity of threats, and the importance and value of the software to your company. Be sure to budget for these activities early on both in cost and time!

SAST—Static Application Security Testing

SAST, or Static Application Security Testing tools, are automated security analysis products that review your source code or byte code looking for security bugs. These security tests are run against "static" source code or byte code, as opposed to testing against a live running application. The benefit of these tools is that they can be run in an automated fashion on a regular basis, they often find vulnerabilities that live testers have limited visibility into, and they reveal the errors directly in code so it is efficient for developers to find the bug in order to fix it. These tools are considered "whitebox testing" tools because they review applications from "the inside out."

For example, you could include static analysis in your nightly CI processes. Any defects could be automatically posted to your bug tracking system and noted in an email sent to the developer. When a developer fixes the bug and checks in the code, the vulnerability can be rescanned and dropped without any intervention from security staff.

However, SAST tools, like any tool, have limitations. SAST tools traditionally create a large number of false positives (they often report security bugs that are not truly exploitable) and have little visibility into business logic (business-specific) vulnerabilities. SAST tools also have a history of taking a very long time to scan large code bases. Many emerging vendors are attempting to fix these issues, but the investment in building effective SAST tools is significant. One of the first Java-based open source SAST tools is the famous FindBugs open source project by Bill Pugh.¹⁵ FindBugs integrates with most Java IDEs as well as several automated build systems, such as Jenkins. This is an excellent and inexpensive way to starting using basic static analysis in a regular and continuous way.

When evaluating a commercial SAST tool to determine what is best for your company and application, consider the Static Analysis Technologies Evaluation Criteria from the Web Application Security Consortium.¹⁶

DAST—Dynamic Application Security Testing

Dynamic Application Security Testing (DAST) tools are essentially "scanners" that attempt a variety of attacks against a live running web application looking for vulnerabilities. These tools,

like SAST tools, are automated scanners that often report a large number of false positives that need to be reviewed by a security professional. DAST tools are often called “black box testing” because they test from the “outside in” without knowledge of the inner working of your software. This is often considered to be the “hacker’s” perspective when testing your application.

One of the greatest ways to make a developer “not care about security” is to give him a security report that is laden with security bugs, which after close inspection, are not really bugs at all (in other words false positives). While these tools can be tuned to report fewer false positives, doing so will also reduce the number of actual real security findings. If you fire a scanner at your application “untuned,” the false positives are often high. If you tune the tool in the other direction, you may miss critical bugs. Not to mention that DAST tools, like SAST, also have difficulty finding business process–specific bugs like access control vulnerabilities.

The good news is that DAST security “findings,” when verified, are not theoretical. DAST findings are real-world exploitable bugs that often need to be fixed right away. SAST and DAST tools review your applications for security bugs in different ways. Both are relevant and should be used together to thoroughly review your application for security.

Manual Penetration Testing

Tools are great, but they are useless without a knowledgeable person to wield them. Everything we do as developers involves custom development, so why should you trust a cookie cutter solution to protect your app? This is where professional security testers (a.k.a. penetration testers or, simply, pentesters) come into play.

Pentesters excel at doing deep investigations of issues, investigating business logic workflows, and verifying the issues discovered by other tools. Pentesters can also chain a variety of flaws together to craft a complete exploit. A complete exploit can help explain the severity of an issue to management in better terms than rote output from a tool. A manager might hear SQL injection and get a glazed look in his eyes, but if he hears “SQL injection helped me exfiltrate your entire database to discover weak password hashes, which were subsequently reversed using a rainbow table, allowing me to log in with an administrator account, and then transfer money to third parties,” that will really get his attention.

Pentesters have their drawbacks, too. Good penetration testers are rare, expensive, and they don’t scale very well. In an environment using a rapid development cycle or “extreme coding,” it can be difficult for a sole pentester to keep up. A combination of DAST, SAST, and knowledgeable pentesters is often best solution for the best use of scarce resources.

Have Monitoring and Response/Recovery Plans in Place

Even if you have followed every suggestion in this book, there is still a strong possibility that at some point your application may be compromised. In fact, it might be compromised already! Before you go live, it is prudent to have a plan in place to deal with a negative security event should such a situation arise. Incident response plans will be different for every organization, but you can start by answering the following questions:

• • • • • Image What do I do if a vulnerability is discovered on my live website?

• • • • • Image What do I do if I discover that I’m actively being hacked?

• • • • • Image Do I have backups in case of data loss?

Image Do I have a failover plan in place in the case of a denial of service?

Image Am I logging enough information to reconstruct how a breach happened, by whom, and what was accessed?

Image What do I do if I discover that the public (as from Pastebin) knows that I was recently hacked?

Image What am I legally required to disclose to my users?

These are the questions that every incident response team must prepare for. Incident response is a huge topic, and could probably be the subject of an entire book in and of itself. For the purposes of this book, incident response teams and developers really intersect in the realm of logging and intrusion detection. We recommend that you review [Chapter 9](#) for more information about logging. During actual operation of your site, be sure to review your log files with your incident response team to make sure that they are being written correctly and backed up, and that you have an appropriate level of information being logged. Also, you should know your log rotation policy and specify an appropriate duration for log retention. In the wake of an event, it is critical to evaluate what worked and what didn't, and then adjust your process accordingly. In the post mortem, you want to make sure that you not only fix the one thing that went wrong, but also fix the entire *class* of problems. So if you found that you were vulnerable to XSS for example, don't just fix the one location that was exploited. Instead, do an application inventory assessment and make sure that you are using proper output encoding in all locations throughout your site. If necessary, you may need to revisit your security controls and make adjustments.

Image

TIP

Chances are that if you have an issue with one application, you may have the same problem with others in your company. The first thing your adversaries will try is the same exploit on another application. At the organization level, you need to ensure that you have the right security controls in place, make sure that project managers understand that fixing the issue is a priority, and make sure that your developers have the appropriate training and resources to fix this issue throughout the organization.

Web Application Firewall (WAF)

WAFs are for wimps. –Jim Routh

Web application firewalls (WAFs) are essentially *layer seven* (the application layer) firewalls that inspect HTTP traffic and attempt to block certain attacks based on clearly defined rules.

WAFs have a very controversial history. Operations folks love web application firewalls because they give them the power to seemingly *secure* applications without depending on software developers to actually fix their code. However, attackers are also fond of applications that depend overmuch on *default configured* web application firewalls because they are often easy to circumvent. WAFs can be detected using tools such as *wafw00f* by Sandro Gauci.¹⁷ Once detected, an attacker can employ a variety of WAF evasion techniques. During an audit years ago, I was attempting to break through a WAF that was blocking my SQL injection attack, *1 or 1=1*. I only had to replace that with *1 or 2=1+1* to evade this rather simple firewall rule.

The essence of the quote from Jim Routh at the start of this section is essential: If WAFs are the only defensive layer protecting your software, then you are not only a secure software “wimp,” you are also likely already compromised. Another way of expressing this idea is that you should emphasize fixing your code whenever possible, rather than attempting to firewall those vulnerabilities away.

While it’s easy to throw WAFs under the bus and drive over them a few times, there are situations in which a web application firewall can be a powerful ally. Suppose you depend on a COTS (Commercial Off The Shelf) web application suite and do not have control of the code. Sometimes your only choice is to either decommission that application or use a carefully configured WAF rule when a security vulnerability is found. Sometimes your developers have other priorities and are not able to fix live vulnerabilities for some time. Even the most agile development organization can’t triage, design, code, test, and deploy a solution fast enough. In such cases, WAFs are a great tool to lock bugs down quickly and “buy you time” until your developers can fix the underlying bugs. WAFs are not perfect security and often have problems. But sometimes WAFs are all you have if you want to keep doing business. Just remember that a WAF should never be a permanent solution.

One of the oldest open source WAFs is the Apache Foundation’s ModSecurity, the open source web application firewall.¹⁸ While this project is part of the Apache Foundation, the OWASP foundation manages the core ruleset. Please visit the OWASP ModSecurity core rule set¹⁹ (managed by Ryan Barnett) for more information.

Â Â Â Â Â Â Â Image

Â Â Â Â Â Â Â **TIP**

Â Â Â Â Â Â Â *Look for integrations between different vendors so you can leverage the work of one approach with another. For example, WhiteHat Security provides a DAST assessment service that feeds into Imperva, F5 ASM, and other web application firewalls.*²⁰

Summary

There are many methodologies to build software. The debate between Agile, Waterfall, DevOps, and other methodologies is a “religious debate” that we will politely avoid. However, regardless of your chosen software building methodology, it’s important to consider security at every level. Understand that the “faster you go” the more “discipline you need” in your security testing activities. Know your threats, assets, and regulatory environment. Design for security from day one! Give your developers the right education and tools to build secure software! Test early and test often! Have monitoring and incident response plans in place! And include the lessons from one development cycle in the next in order to constantly improve your security posture.

Closing Thoughts

When you care about security in your software, everybody wins. The developer sitting next to you wins because you become a resource to help your team write more secure code. Your company wins because it is able to release robust and secure software. Your customer wins because you can successfully protect their personal data. Security engineering also protects the intangible—the hopes and dreams of your users. They depend on you. They depend on your website for email, for managing their finances, for health care information, for communication with their loved ones, and recreation. The Web and other networked applications permeate all aspects of our lives more and more every day.

Finally, caring about security engineering will benefit you, the Java developer. Thinking critically about security will help open your mind to new ways of approaching software development. Security engineering is an important skill that not enough developers have mastered. The world needs more developers who care and master security engineering. There is a great need to “rebuild the Web” in a secure fashion and that starts with you and the next line of code you write.

This book is not an end, but rather the beginning of your mastery of application security and security engineering. We are grateful that you took these first steps with us.

Image

¹ <https://twitter.com/erratarob/status/469302869712060418>

² Perhaps “mushroom” would be a more apt description.

³ https://www.owasp.org/index.php/OWASP_Code_Pulse_Project

⁴ <http://www.bsimm.com/> and <http://www.opensamm.org/> respectively.

⁵ Articles 23, 24, and 79 of the “Administrative sanctions” section of <http://register.consilium.europa.eu/pdf/en/12/st05/st05853.en12.pdf>

⁶ https://www.owasp.org/index.php/OWASP_Secure_Coding_Practices_-_Quick_Reference_Guide and https://www.owasp.org/index.php/Category:OWASP_Application_Security_Verification_Standard_Project

⁷ <http://shiro.apache.org/>

⁸ www.oracle.com/technetwork/java/index-jsp-135995.html; https://www.owasp.org/index.php/OWASP_Java_Encoder_Project; https://www.owasp.org/index.php/OWASP_Java_HTML_Sanitizer_Project; and https://www.owasp.org/index.php/OWASP_JSON_Sanitizer

⁹ www.keyczar.org/

¹⁰ <https://www.bouncycastle.org/>

¹¹ <https://github.com/sourceclear/headlines>

¹² <http://logback.qos.ch/>

¹³ <https://www.aspectsecurity.com/uploads/downloads/2012/03/Aspect-Security-The-Unfortunate-Reality-of-Insecure-Libraries.pdf>

¹⁴ https://www.owasp.org/index.php/OWASP_Dependency_Check

¹⁵ <http://findbugs.sourceforge.net/>

¹⁶ <http://projects.webappsec.org/w/page/66094278/Static%20Analysis%20Technologies%20Evaluation%20Criteria>

¹⁷ <https://code.google.com/p/waffit/source/browse/trunk/wafw00f.py>

¹⁸ <https://www.modsecurity.org/>

¹⁹ https://www.owasp.org/index.php/Category:OWASP_ModSecurity_Core_Rule_Set_Project

²⁰ Jim is a former employee of WhiteHat Security so we may be a bit biased here. There are many security vendor solutions; it is wise to review all of the available offerings before making serious purchasing decisions.